Bat Speed Across the Swing from Broadcast Video

Using an unpaired library of sensor-recorded shots

str8bat AI Engineer Technical Take-Home · Part 1: System Design · Part 2: Prototype on real footage

Devansh Mittal

Summary. Broadcast video tells us *where* the batter's hands and bat are, roughly and in 2D, about 25 times a second. The sensor library tells us *what real swings look like*: dense, 3D-consistent, 100-sample bat-speed curves for millions of shots. Neither is enough alone, and they are not paired. My design bridges them through **simulation**: sensor swings are rendered into synthetic broadcast-style 2D observations, which gives unlimited paired training data, and the real clip is then explained by the most likely sensor-like swing that reprojects onto what the camera saw. Metric scale comes from regulation pitch geometry, not from the prior. The output is a 100-sample bat-speed curve $v(\tau)$, an impact speed, and an uncertainty band. Part 2 runs the observation front-end on a real 72-frame clip: wrist tracking, a velocity profile in px/s and km/h, and a frame-by-frame account of where tracking broke (74% of shot frames were flagged) and why.

1 Part 1: System design

1.1 What the system must produce, and what makes it hard

For one shot in a broadcast clip, the target is the same quantity the sensor records: bat speed at each of ∼100 samples across the swing, $v(\tau)$ for $\tau = 1, \ldots, 100$, plus impact speed $v(\tau_{\text{imp}})$. Four properties of broadcast video make naive pixel differentiation fail:

- **No calibration, moving camera.** The operator pans and zooms, and the director cuts. Pixel motion mixes batter and camera.
- **Depth loss.** The main camera looks down the pitch, so a straight drive moves largely *along* the line of sight and looks slow in 2D.
- **Too few frames.** A downswing lasts about 150–200 ms: 4–5 frames at 25–30 fps, and some of those may be pulldown duplicates (Part 2 found one exactly at impact).
- **The bat is the hardest thing to see.** It is thin, fast and blurred at peak speed, and often hidden by the body in stance and follow-through.

1.2 Key assumptions (made explicit)

A1 Sensor content. Each record has per-sample bat speed and impact speed. I assume it *also* has bat orientation or angular rate (typical for a bat-mounted IMU). If it only has speed, the prior is learned on speed curves alone and the swing plane comes from video (Section 1.6).

A2 Known sample timing. The 100 samples have a consistent definition: either a fixed rate, or a phase-normalised window with an identifiable impact index. Without this, video time cannot be mapped onto τ.

A3 Standard geometry. Stumps are 0.711 m tall and the crease markings are regulation, so they are a free calibration target. Bat length is about 0.85 m, and the sweet spot is 0.55–0.65 m from the top hand.

A4 Shape transfers, scale may not. The sensor library (probably mostly amateur and club players) covers the *shape* of elite swings, but not necessarily their *magnitude*.

A5 Usable footage. The batter is in shot from stance to early follow-through on the main camera, at 25–50 fps, at least 720p.

1.3 Architecture

Figure 1 shows the end-to-end system. The blue path runs per clip. The green path runs offline, once, on the sensor library. They meet in the fusion stage (purple).

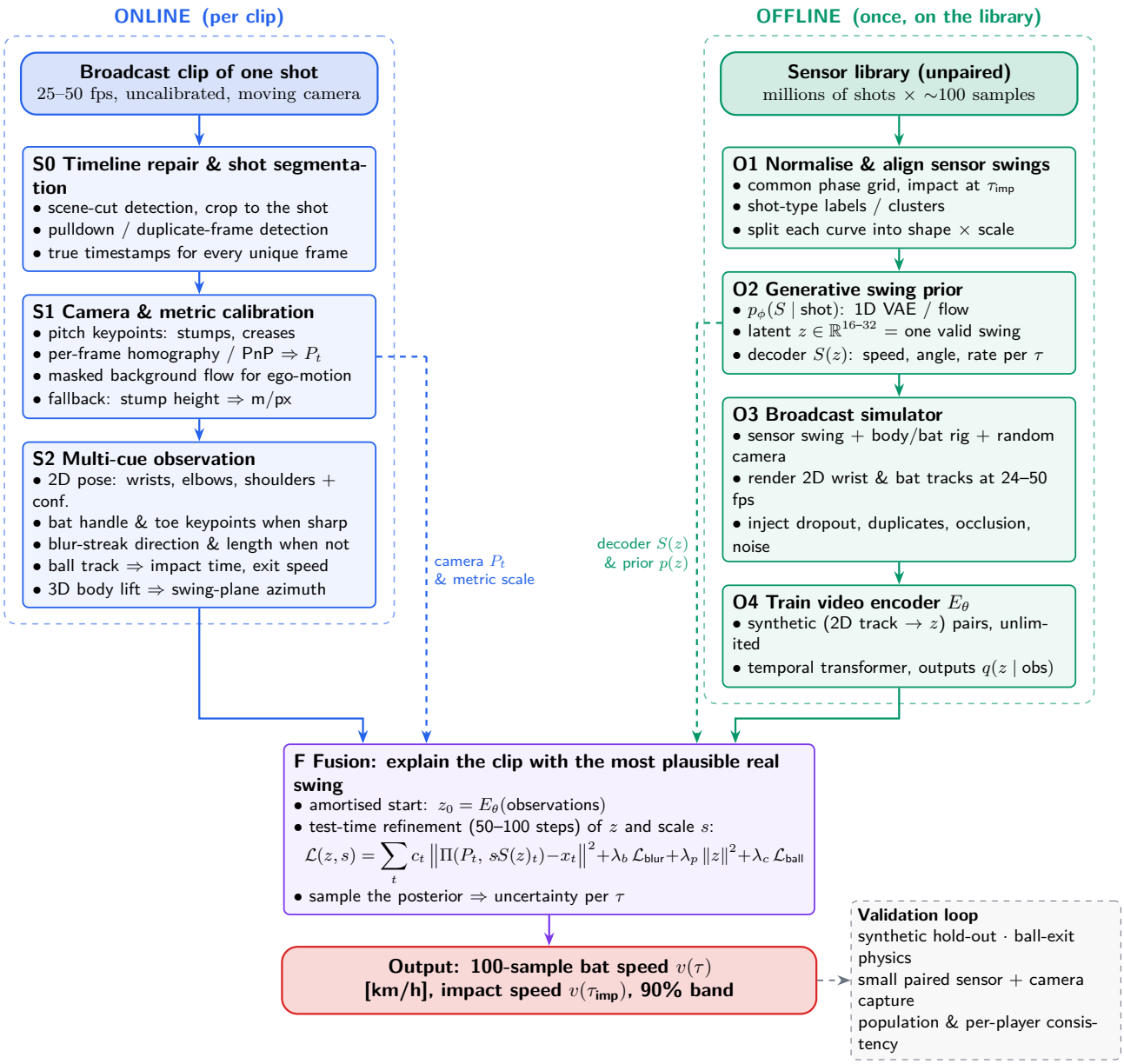


Figure 1: End-to-end architecture. The two lanes never need paired data: the simulator (O3) manufactures it from the sensor library, and the fusion stage (F) only asks which realistic swing best explains the observed 2D evidence. Calibration (S1) fixes metric scale *independently* of the prior, so elite speeds are not pulled down towards the library average.

1.4 What I would build, stage by stage

S0 Timeline repair. Detect scene cuts, isolate the batting shot, and detect duplicate frames from inter-frame difference. This was not in my first design. Part 2 showed it matters: the clip's active section is 24 fps material in a 30 fps stream, so every 5th frame is a copy, *including the impact frame*. Each unique frame gets its true timestamp, and duplicates are removed rather than differentiated.

S1 Calibration. A small keypoint model finds stump tops and bases and crease corners. With 4 or more planar points plus the vertical stump, solve a per-frame PnP for $P_t = K_t[R_t \mid T_t]$. When the creases are not visible, propagate P_t with background optical flow (players masked out). The minimum fallback is the stump-height scale at the crease plane. That is what Part 2 uses: 99.25 px = 0.711 m, so 7.16 mm/px.

S2 Observations. These are the cues that survive broadcast conditions. (i) A top-down pose model on a stabilised batter crop gives wrists, elbows and shoulders with confidences. Wrists move about 3× slower than the sweet spot, so they stay sharp when the blade blurs. (ii) A bat keypoint model (handle, toe) for low-speed frames. (iii) A blur-streak head for high-speed frames. It gives the blade

direction reliably and a *relative* length, because broadcast exposure time is unknown. (iv) A ball tracker, for impact time from the deflection vertex and exit speed. (v) Monocular 3D body lifting, for the swing-plane azimuth.

O1–O4 Sensor side. Align every sensor swing to a common phase grid anchored at impact. Factor it into a normalised *shape* $v(\tau)/v_{\max}$ and a *scale* $v_{\max}$. Fit a conditional generative prior over shape. Then build the bridge: a simulator places a decoded swing on a parametric body-and-bat rig, picks a random broadcast camera (position, focal length, pan), renders 2D wrist and bat tracks at the clip's frame rate, and corrupts them the way Part 2 saw real data corrupted (dropped trail wrist, duplicate frames, label swaps, pan drift). That gives unlimited (observation, ground-truth $v(\tau)$) pairs for training the encoder E_θ.

F Fusion. E_θ gives a fast first guess. A short test-time optimisation then refines the latent swing z and scale s so that the projected swing matches the observed keypoints, weighted by confidence. It respects blur and ball-exit evidence and stays on the prior. Sampling the posterior gives a band at every τ, which widens where the video is blind.

Data relationships exploited.

- **Rigid-body lever arm:** $\vec{v}_{\text{bat}} = \vec{v}_{\text{wrist}} + \vec{\omega}_{\text{bat}} \times \vec{r}$, with $|\vec{r}| \approx 0.55$–$0.65\,\text{m}$. Visible wrists and forearm rotation constrain the bat while it is invisible.
- **Low-dimensional swing manifold:** real $v(\tau)$ curves are smooth and stereotyped: backlift, accelerating downswing, peak just before impact, a dip at impact, then decay. A handful of noisy frames is enough to pick the right curve from this family.
- **Shot type ↔ swing plane:** drives, pulls and cuts differ in plane, timing and peak ratio. The shot class conditions the prior and resolves much of the depth ambiguity.
- **Ball exit ↔ impact speed:** for a middled shot, $v_{\text{out}} \approx e_A\, v_{\text{in}} + (1 + e_A)\, v_{\text{bat}}$, with collision efficiency $e_A \approx 0.1$–0.2 for a cricket bat. Ball speeds in and out are measurable from the video, so this gives an independent check on impact speed.
- **Pitch geometry ↔ metric scale:** regulation stumps and creases give m/px without any calibration rig.

Why video samples are not enough on their own. Figure 2 shows the core argument for using the sensor prior. At broadcast rates the peak usually falls between frames, and finite differences across 33–42 ms flatten it.

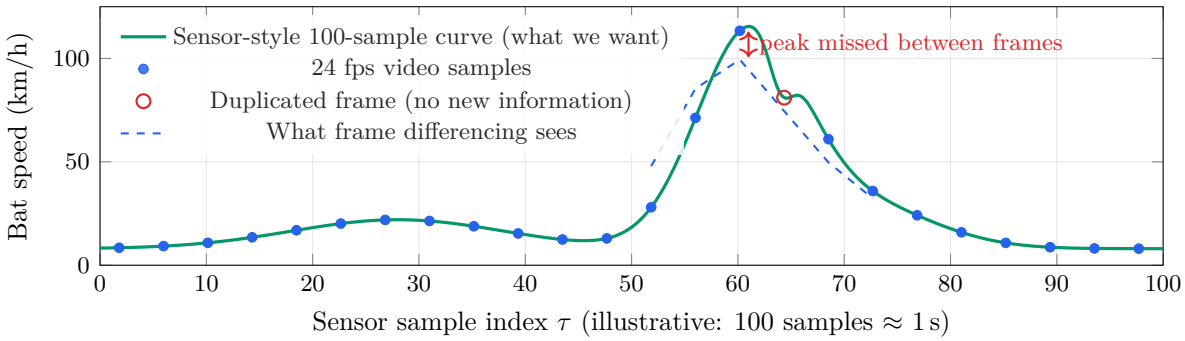


Figure 2: Illustrative (not measured). With ∼4 sensor samples per video frame, the true peak and the impact dip fall between frames, and a duplicate frame at impact doubles the gap. Fitting the sensor-learned shape through the video samples recovers the curve between them. Interpolating the samples directly cannot.

1.5 Build order

1. **Baseline (weeks 1–2):** S0 + S1 (stump scale) + S2 pose. Regress a single shape template from the sensor library through the observed wrist speeds. This is already better than raw differentiation, and it is the slice Part 2 proves.

2. **Prior + simulator (weeks 3–6):** O1–O4. Train E_θ on synthetic pairs and test on held-out synthetic swings.
3. **Full fusion (weeks 6–10):** PnP calibration, bat and blur heads, ball-exit term, test-time optimisation, uncertainty.
4. **Ground truth (in parallel):** a small paired capture session (see Validation). It is the single most valuable dataset for this problem.

1.6 Hardest risks

Risk	Why it bites on broadcast footage	Mitigation
Depth ambiguity	Down-the-pitch camera: straight drives move along the optical axis. In Part 2 a guessed 50° angle changes speed by 31%.	Swing-plane azimuth from 3D body lift, footwork and ball exit direction. Shot-conditioned prior. Report the band, not just a point.
Temporal aliasing & pulldown	4–5 frames per downswing, and 1 in 5 frames can be a duplicate. Finite differences under-read the peak.	S0 removes duplicates. The sensor shape is fitted through the samples instead of interpolating between them.
Bat invisible at peak	Blur and occlusion in exactly the frames that matter.	Lever-arm coupling to wrists. Blur direction. Prior fills the gaps and the uncertainty widens there.
Sim-to-real gap	The encoder is trained only on simulated tracks.	Corrupt synthetic tracks with noise statistics measured on real clips (Part 2 flags). Test-time optimisation against real evidence.
Scale shrinkage	An amateur-heavy library pulls elite speeds down.	Prior on *shape* only. Scale comes from geometry and ball exit. Check per-player bias.
Unknown sensor content (A1/A2)	If only scalar speed, or an unknown time base, the alignment weakens.	Settle this first with the data owners. Fall back to a speed-only prior plus impact anchoring.

1.7 How we would know the estimate is any good

V1 Synthetic hold-out: render held-out sensor swings through the simulator with unseen cameras. Report per-sample RMSE, impact-speed MAE, peak-timing error and 90%-band coverage. This tests the method, not realism.

V2 Paired capture (decisive): film 200–500 shots by sensor-equipped batters with broadcast-like cameras (behind the bowler, 25–50 fps, zoom and pan). This is the only true end-to-end test. It also calibrates the uncertainty.

V3 Physics checks on real broadcast: predicted impact speed against ball-exit speed via e_A. Wrist-to-sweet-spot ratio within the physical range.

V4 Population checks: for each shot type, the predicted distribution across many broadcast shots should sit plausibly against the sensor distribution. The same batter's repeated shots should be consistent, and harder shots should rank higher.

Target (assumption): impact speed within ±8 km/h, peak timing within ±1 frame, and 90% bands that contain the truth about 90% of the time.

2 Part 2: A working slice on real broadcast footage

2.1 Clip and setup

A public broadcast clip of Liam Livingstone (England v Pakistan T20, main camera behind the bowler) playing a lofted on-side drive: 720×1280 portrait, 29.95 fps, 72 frames (2.40 s). Frames 0–21 are a different camera shot before a cut. The batting shot is frames 22–71. This slice implements S0, S1 (stump-scale fallback) and S2 (pose) from Figure 1, plus a lever-arm bat proxy.

1. **S0:** frame extraction, cut detection, duplicate detection (mean absolute grey difference < 1.5).
2. **S1:** Lucas–Kanade flow on background features with YOLOv8 people masks, partial-affine RANSAC for camera $\Delta x, \Delta y$, and stump-height scale.
3. **S2:** YOLOv8x-pose (COCO-17) in a striker corridor ROI. Wrist gating and imputation. Camera-motion subtraction. PCHIP interpolation over unique frames only. Savitzky–Golay derivatives.
4. **Bat proxy:** blade direction from the forearm vector refined by an image-gradient search (not a bat detector), then $\vec{v}_{\text{bat}} = \vec{v}_{\text{pivot}} + \omega r\, \hat{n}_{\perp}$ with $r = 0.55$ m.

Code: `run_pipeline.py`, `src/camera_motion.py`, `src/track_swing.py`, `src/plot_velocity.py`.



Figure 3: Frame 56 (top of frame). The batter is about 150 px tall, and the stumps used for scale are 99 px.

Table 1: Calibration and clip facts.

Quantity	Value	How obtained
Resolution / frame rate	720×1280 / 29.954 fps	container ($\Delta t = 33.4$ ms)
Frames / duration	72 / 2.404 s	shot window: frames 22–71 (50 frames)
Duplicate frames	14 in clip, 10 in shot	every 5th frame: 23, 27, 32, …, 57, 62, 67
Effective source rate	≈24 fps	1 duplicate per 5 frames ⇒ 2:3 pulldown
Stump height in image	99.25 px	top (340, 809), base (333, 908)
Scale at crease plane	0.007164 m/px (7.16 mm/px)	0.711 m / 99.25 px
px/s → km/h	× 0.02579	0.007164 × 3.6
Camera speed in shot	mean 15.7, max 140.3 px/s	frames 29–31, delivery pan

2.2 Velocity profile across the swing

Figure 4 is the main result. It is plotted directly from `swing_kinematics.csv`. The raw (grey) curve is the naive frame-to-frame handle speed. The green curve is after duplicate removal, camera compensation and wrist gating. The red and purple curves are the bat-proxy sweet spot in 2D and with the out-of-plane correction.

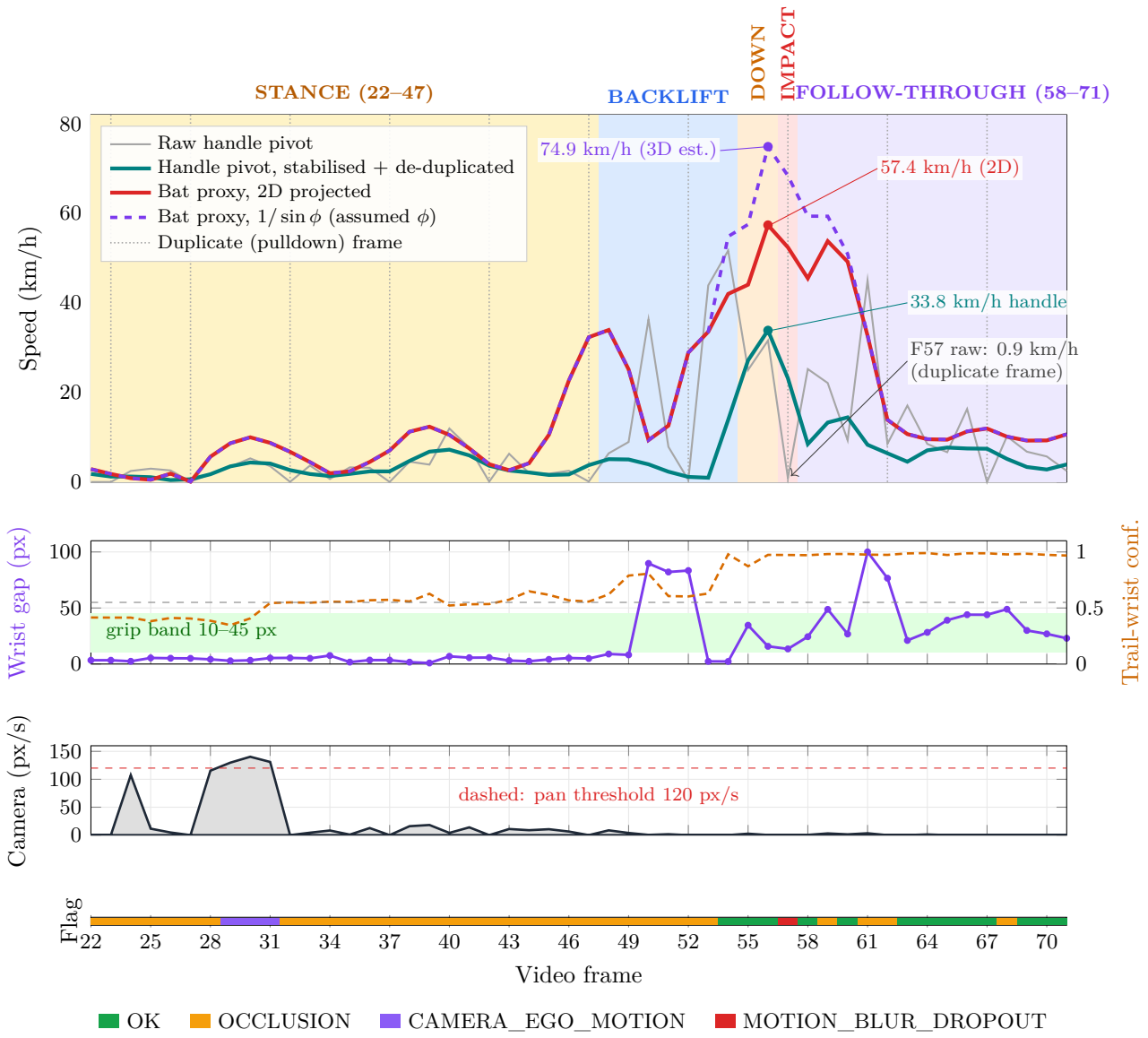


Figure 4: Velocity profile across the swing (frames 22–71), plotted from `swing_kinematics.csv`. **Top:** speed in km/h (px/s = km/h / 0.02579). Dotted verticals mark duplicate frames. **Second:** inter-wrist distance (purple, left axis) against the plausible grip band, and trail-wrist confidence (orange dashed, right axis; grey line is the 0.55 gating threshold). **Third:** background camera speed. **Bottom:** per-frame tracking flag.

Table 2: Kinematic results (shot window, frames 22–71).

Metric	px/s	km/h	Frame / note
Peak handle-pivot speed (stabilised)	1312	33.8	F56, late downswing
Peak bat sweet-spot proxy, 2D projected	2224	57.4	F56
Peak bat proxy, $1/\sin 50°$ corrected	2903	74.9	F56; ϕ assumed, not measured
Bat-to-handle ratio at peak		1.70×	lever-arm contribution ≈ +23.5 km/h
Bat proxy at impact window	2031	52.4 (68.4 corr.)	F57 is a duplicate of F56
Raw pivot speed at F57	34	0.9	artefact of duplicate frame
Raw pivot peak (uncorrected)	2003	51.7	F54: wrist label jump, not motion

2.3 What it takes to get real-world units

The km/h axis above uses one scale factor: the stump height at the crease plane. That is a first-order conversion. A trustworthy one needs four more things:

1. **Depth-correct scale.** The batter stands about 1.2 m in front of the stumps, closer to the camera,

so 7.16 mm/px is slightly wrong for the hands. A full PnP from crease corners plus the vertical stump gives scale at the batter's actual depth.

2. **The out-of-plane component.** A down-the-pitch camera cannot see motion along its axis. I applied $1/\sin\phi$ with $\phi = 50°$ in the downswing, which adds 31%. That angle is a modelling assumption. It should come from the swing plane (3D body lift and ball direction), or better, from the sensor prior.
3. **Sub-frame timing.** At an effective 24 fps with the impact frame duplicated, frames 56 to 58 span 67 ms. The real peak is sharper than any finite difference can show (Figure 2).
4. **Actual bat tracking.** The sweet-spot speed comes from a forearm-direction proxy and an assumed 0.55 m lever. A bat keypoint or blur model would replace the assumption with a measurement.

The 74.9 km/h estimate is well below the 100 km/h-plus typical of an elite power hit. I read that gap as a measure of what these four corrections are still missing, not as a finding about the batter.

2.4 Where tracking failed, and why

Of the 50 shot frames, 13 (26%) tracked cleanly. Figure 5 shows each failure type and Table 3 counts them.

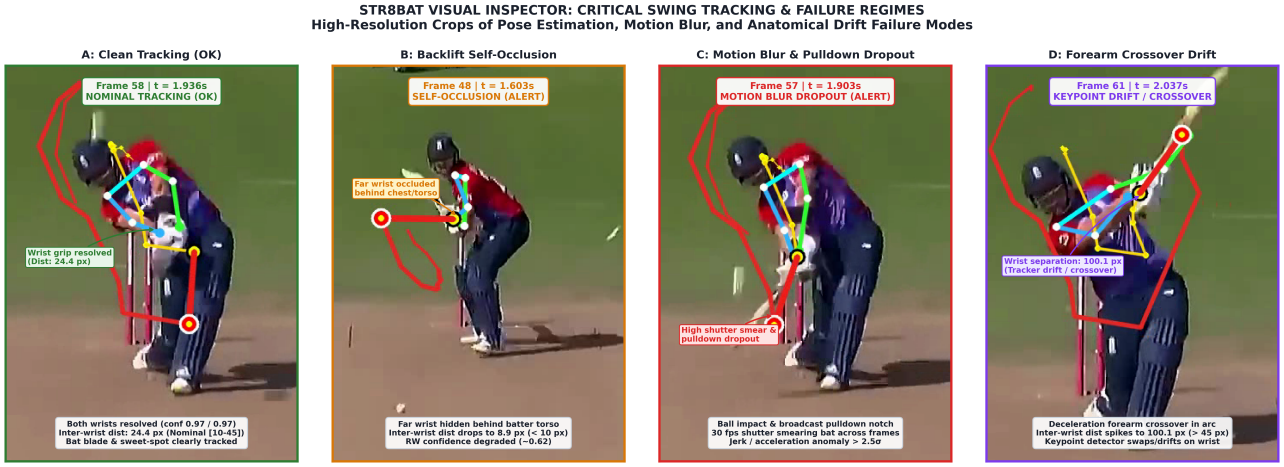


Figure 5: Pipeline output on four representative frames. **A** (F58): clean two-wrist tracking. **B** (F48): trail wrist hidden behind the torso. **C** (F57): impact frame, which is a pulldown duplicate with blade smear. **D** (F61): forearm crossover in follow-through. Wrist labels drift and the separation jumps to 100 px.



F55, t=1.836 s F56, t=1.870 s F57, t=1.903 s (= F56) F58, t=1.936 s

Figure 6: The impact window, uncropped in time. F57 is pixel-identical to F56 (mean absolute difference 0.15 grey levels), so ball–bat contact is never seen. It happens somewhere in the 67 ms between F56 and F58. Note also how small the bat is at this resolution.

Table 3: Failure flags across the shot window (50 frames), from `swing_kinematics.csv`.

Flag	Frames	%	Where	Why it broke
OCCLUSION	33	66	22–28, 32–53, 59, 61–62, 68	*Stance/backlift:* the trail wrist is behind the lead forearm and chest. Confidence 0.35–0.65. The model puts both wrists on the visible hand (gap <10 px), or the trail wrist jumps to the elbow or bat top (90 px at F50). *Follow-through:* the forearms cross, the labels swap or drift onto shoulder or helmet, and the gap reaches 100 px at F61.
CAMERA_EGO_MOTION	3	6	29–31	The camera pans with the bowler's delivery at 130–140 px/s. Without background-flow subtraction this would add phantom batter speed.
MOTION_BLUR_DROPOUT	1	2	57	Impact frame is a pulldown duplicate, and the blade is smeared. Raw pivot speed drops from 31.6 to 0.9 km/h.
OK	13	26	54–56, 58, 60, 63–67, 69–71	Both wrists detected with confidence above 0.85 and no implausible jump in wrist position.

2.5 What did not work (honest note)

- **The bat itself was never detected.** The "bat speed" is a physics proxy: wrist motion, forearm direction and an assumed lever. At about 150 px batter height the blade is a few pixels wide and blurred, and off-the-shelf detectors do not find it.
- **Most frames are not clean.** 74% of shot frames were flagged. The trail wrist is imputed from the lead wrist for most of the stance, so stance-phase speeds rest on one wrist.
- **The peak sits in the worst part of the clip.** The downswing has 3 unique frames, and the impact frame is a duplicate. The smoothed peak depends on the spline and Savitzky–Golay window choices. With a 5-frame window across a 3-frame event, it is a lower bound, not a measurement.
- **The 3D correction is a guess.** $\phi = 50°$ is a judgement from watching the shot, so the 74.9 km/h figure carries that assumption.
- **Gating can hide real motion.** The rules that reject wrist spikes (gap >45 px, confidence <0.55) were tuned by eye on this clip and need validation on others.
- **What worked:** duplicate detection, which explained the impact-frame dropout. Camera-motion subtraction during the delivery pan. A failure classifier whose flags match what is visibly wrong in the frames. These map directly onto S0–S2 of the design and onto the noise model the simulator (O3) needs.

2.6 Reproducing

```
pip install -r requirements.txt
python run_pipeline.py                                   (about 50 s on a laptop)
```

Outputs in `outputs/`: `swing_kinematics.csv` (72 rows × 32 columns), `camera_metrics.json`, `annotated_swing.mp4`, `velocity_profile_analysis.png`, `failure_frames_strip.png`.

A Appendix: original diagnostic figure

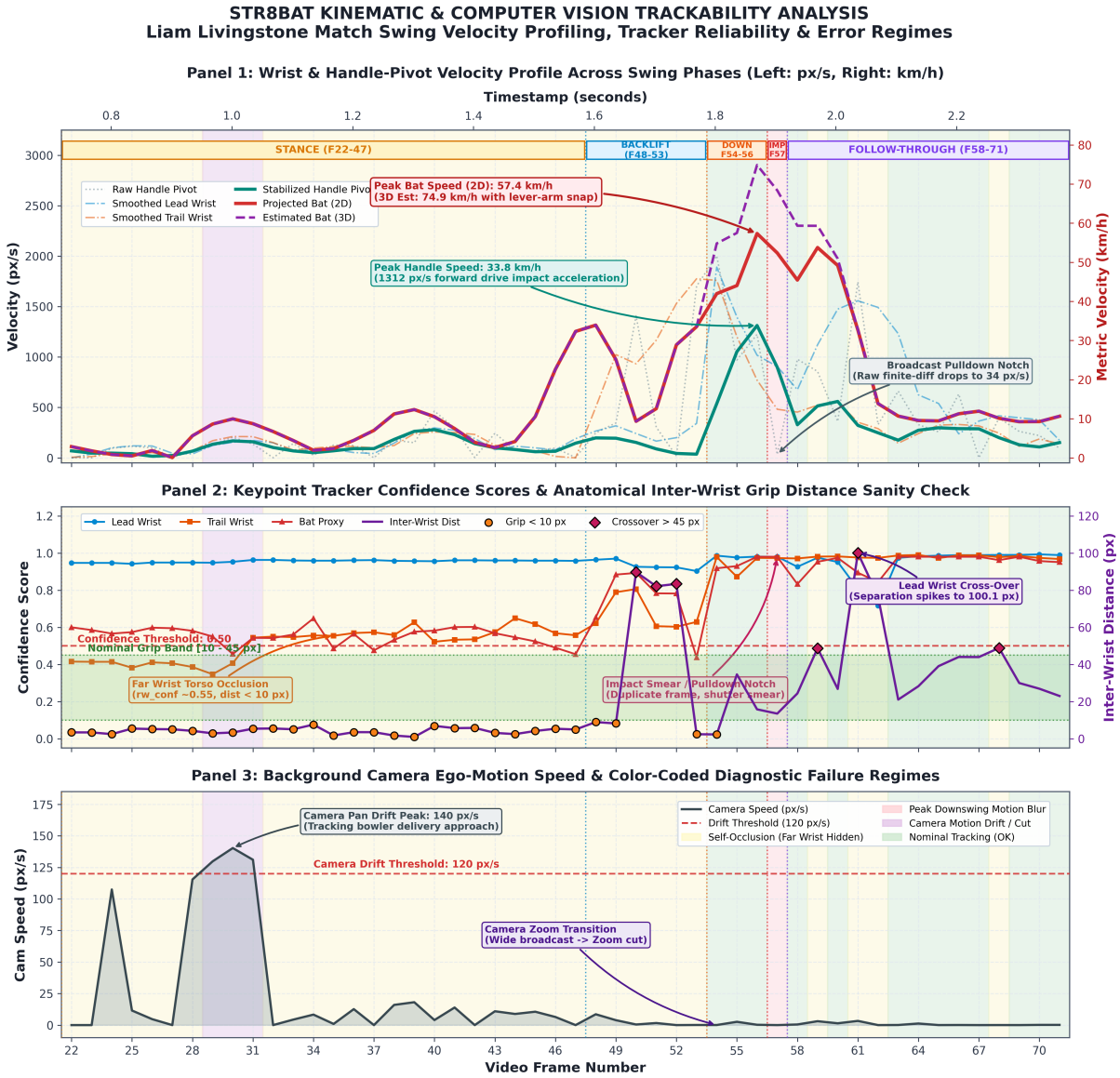


Figure 7: The pipeline's own 3-panel diagnostic figure (`outputs/velocity_profile_analysis.png`), kept for reference. Figure 4 redraws the same data.